

Computation II

Algorithms & Data Structures

Project Guidelines

Paulo Souza, Pedro Fernandes and Diogo Rasteiro

March, 2026

1 Project Rules

This project aims to evaluate the knowledge and skills acquired throughout the semester, particularly your ability to apply algorithms, data structures, and analyze the underlying running time complexity in the design and implementation of a puzzle game inspired by *Bloxorz*.

- **Group composition** — Students must work in groups of 4 or 5 members (mandatory). The group composition must be submitted on Moodle via a specific activity that will be made available soon.
- **Deadline** — The final project must be submitted by **May 17th, 23h59**. Late submissions will incur a penalty of **2 points per day**. Projects submitted more than 3 days late will not be considered for evaluation.
- **Submission** — You must submit your project code, a short accompanying report, and a self-evaluation spreadsheet. Submission must be made through Moodle. A detailed description of the required deliverables will be provided in Subsection 3.5 and Section 4 of the present document.

- **Defense** — All projects must be defended in person. Each group will be required to answer questions about their design decisions, complexity analysis, and code implementation. Be prepared to answer *technical* questions. A link to schedule your defense time slot will be provided beforehand. **Failure of a member to attend the defense will result in their project grade being 0.**
- **Minimum Grade** — This project has a minimum grade of **8 out of 20**. People with projects that do not meet this threshold will not be eligible for the first call season.

2 Project Description

The main goal of this project is to implement a version of the puzzle game *Bloxorz* (also known as *Space Block*). In this game, the player controls a rectangular $2 \times 1 \times 1$ block on a grid-based board and must navigate it to fall upright into a designated goal hole, using as few moves as possible.

Students should implement the necessary classes and functions to ensure the game's core functionality. Students should also analyze the overall complexity and performance of their implementation, including the search algorithms used to solve levels.

Additionally, a **solver** must be implemented: given a board configuration and a starting block position, the solver automatically searches for a sequence of moves that brings the block to the goal tile. The solver treats the puzzle as a **graph search problem**, where each distinct configuration of the block on the board (its position and orientation) constitutes a *state*, and each valid move corresponds to a transition between states. The solver must explore this state space and return the sequence of moves corresponding to a path from the initial state to the goal state, if one exists. Further details on the solver requirements are given in Section 3.2.

The game must run in the Python console, reading levels from a matrix representation, accepting directional inputs from the user, and detecting win and loss conditions.

The Block

The block occupies either one or two cells of the board at any time, depending on its orientation:

- **Standing upright** — the block occupies exactly **one cell**. This is the only orientation from which the block can fall into the goal hole and win the level.
- **Lying flat (horizontal)** — the block is tipped on its side and occupies **two adjacent cells** in the same row.
- **Lying flat (vertical)** — the block is tipped on its side and occupies **two adjacent cells** in the same column.

At each step, the player may move the block in one of four directions: **up**, **down**, **left**, or **right**. Rolling the block changes both its position and orientation according to the following mechanics:

- **Upright (1×1) block:** Rolling in any direction causes the block to tip over and become flat. Moving horizontally (left/right) produces a **horizontal** (1×2) flat block; moving vertically (up/down) produces a **vertical** (2×1) flat block. In both cases, the leading cell of the new flat block is placed in the adjacent tile in the chosen direction, and the trailing cell occupies the tile the block just left.
- **Flat horizontal (1×2) block:**
 - Moving **left or right**: the block slides in that direction, maintaining its horizontal orientation. Both cells shift one tile in the chosen direction, and the block remains 1×2 .
 - Moving **up or down**: the block stands upright. Both cells fold onto the single adjacent tile in the chosen direction, producing a 1×1 upright block.
- **Flat vertical (2×1) block:**
 - Moving **up or down**: the block slides in that direction, maintaining its vertical orientation. Both cells shift one tile in the chosen direction, and the block remains 2×1 .

- Moving **left or right**: the block stands upright. Both cells fold onto the single adjacent tile in the chosen direction, producing a 1×1 upright block.

If any part of the block lands on an empty cell (0) or outside the board boundaries after a move, the block falls and the level is lost.

The Board

The board can be represented as a 2D matrix of integers, where each cell encodes the type of tile at that position. The possible cell values are described in Table 1.

Cell Value	Meaning
0	Empty cell — the block cannot occupy this position. If any part of the block moves here, it falls off and the level is lost.
1	Normal tile — the block may stand or lie on this cell safely.
2	Starting position — the block begins the level standing upright on this tile.
9	Goal tile — the block must be standing upright on this tile to complete the level.

Table 1: Board cell values and their meanings.

As an example, consider the following board matrix:

```

1 board = [
2     [1, 1, 1, 0, 0, 0, 0, 0, 0, 0],
3     [1, 2, 1, 1, 1, 1, 0, 0, 0, 0],
4     [1, 1, 1, 1, 1, 1, 1, 1, 1, 0],
5     [0, 1, 1, 1, 1, 1, 1, 1, 1, 1],
6     [0, 0, 0, 0, 0, 1, 1, 9, 1, 1],
7     [0, 0, 0, 0, 0, 0, 1, 1, 1, 0],
8 ]

```

Listing 1: Example board matrix.

In this board, the block starts upright at position (`row=1`, `col=1`), marked by 2. The goal is cell (`row=4`, `col=7`), marked by 9. All cells marked 1 are valid tiles, and cells marked 0 are gaps. Stepping off any edge or onto a 0 cell causes the block to fall and the round is lost.

In summary, your code must perform the following steps:

1. Load a level from a board matrix and instantiate the board and block objects.
2. Display the board state in the console, showing the block's current position and orientation.
3. Accept a directional move from the player (`up`, `down`, `left`, `right`).
4. Update the block's position and orientation according to the rolling mechanics.
5. Check for win, loss, or continuation:
 - (a) **Win**: the block is upright and on the goal tile (9).
 - (b) **Loss**: any part of the block is on an empty cell (0) or out of bounds.
 - (c) **Continue**: otherwise, repeat from step 2.
6. On win or loss, display the outcome and prompt the player to replay the level, move to the next level, or exit.
7. Additionally, a **solver mode** must be available: using a search algorithm of your choice (see Section 3.2), automatically find and display a sequence of moves that solves the current level.

Additionally, you are invited to implement extra functionality beyond the core requirements, in exchange for bonus points (see Section 3.3).

3 Project Requirements

3.1 Suggested Classes

Below is a list of classes you are strongly encouraged to implement. These are suggestions, not hard constraints—what matters is that all core requirements are fulfilled and your design is well-structured.

- Block
- Board
- Game

3.2 Search Algorithms

A core algorithmic requirement of this project is the implementation of a **solver**: given a board and a starting block position, the solver must find a sequence of moves that brings the block to the goal, if one exists.

To implement the solver effectively, you must think carefully about what constitutes a **state** in this problem. A state s fully describes a configuration of the block on the board at any point in time. The information is needed to determine which moves are valid and what state each move leads to. Your implementation should represent states explicitly (e.g., as tuples, objects, ...), so that the solver can track visited states, avoid revisiting them, and reconstruct the solution path.

The state space of the puzzle is the set of all reachable states from the initial configuration. The solver must explore this space — typically by building a search tree or graph — until it finds the goal state (block upright on the goal tile) or determines that no solution exists.

You must implement **two search algorithms** from scratch (i.e., without relying on external graph libraries). Examples include BFS, DFS, Greedy best-first search, A^* , and others.

For A^* , you must define and justify a heuristic $h(s)$ for a state s . A natural choice is the Manhattan distance from the block's current position to the goal tile — but you are free to design your own, provided it is *admissible* (i.e., it never overestimates the true cost to reach the goal).

The solver must output the sequence of moves found. Your code should include a prompt allowing the player to choose between playing manually or using the solver, and — before the solver runs — to choose which algorithm to use, so that runtimes of the algorithms and solution quality can be compared.

3.3 Bonus Functionality

Up to **20% of the total project grade** is reserved for bonus features. These are evaluated based on ambition, correctness, and polish. Ideas include:

- **Improved UI** — use `colorama` or `curses` to render the board with colours, block symbols, and animated moves rather than plain text.
- **Special tile types** — implement additional tile mechanics, such as:
 - *Fragile tiles* (3): can only support the block when it is lying flat; they break if the block stands upright on them.
 - *Teleport tiles*: when the block stands on a designated tile, it is instantly transported to another position.
 - *Split tiles*: cause the block to split into two independent half-blocks.
- **Multiple levels** — define and load a sequence of levels with increasing difficulty, allowing the player to progress through them.
- **Move counter and scoring** — track and display the number of moves per level and compare against a par score.
- **Replay / undo** — allow the player to undo the last move or replay the solution step by step.
- **Level editor** — allow the player to design and save their own board layouts.

Any extra features that do not interfere with the core requirements are acceptable. Clearly document what bonus features you implemented in your report.

3.4 Documentation

One of the most important skills a programmer can have is the ability to properly document their code. As such, one of the evaluation components will be the readability and clarity of your code. This includes:

- How descriptive and accurate the names of your variables, functions, and classes are.
- The quality and quantity of inline comments in your code.
- The presence and quality of docstrings for all classes and non-trivial functions.

A single documentation style must be used **consistently throughout all of your code**. Choose one style (e.g., Google style, NumPy style, or reStructuredText) and apply it uniformly to every docstring in the project. Mixing styles, or having some functions documented in one format and others in another, will be penalized. Ensure that your code is readable to an outsider and easy to follow.

3.5 Report

Alongside your code, you must submit a brief report of at most **4 pages** of content (not including the cover page). The report should contain at least the following:

- A brief overview of your code's organization, including the classes created and the major functions.
- An explanation of the key design decisions you made, such as how you represent the block's state, how you encode the board, and how you implemented the rolling mechanics.
- An analysis of the solvers: how they were implemented, what heuristic you chose for A*, and how the two algorithms compare in terms of performance and solution quality on your test levels.
- An estimation of the **time and space complexity** of both search algorithms in terms of the board dimensions and the number of reachable states.

- Brief instructions on how to run your project (which file to execute and any dependencies).
- A description of any bonus features implemented.
- Any additional information you find relevant.

We are interested in *why* you made the choices you did, not a line-by-line description of your code. Do not pad the report to reach the page limit: quality over quantity.

The following deliverables must be submitted:

- All files required to run your project.
- Your report in PDF format.
- A self-evaluation spreadsheet in `.xlsx` format (see Section 4 for details).

Important Note: We must be able to run your project by executing a single Python file. It is your responsibility to ensure the submitted code works. If we are required to fix the code in order to run it, your grade will be penalized. Always test your submission before uploading.

4 Self-Evaluation Spreadsheet

Each group must submit a self-evaluation in the form of an `.xlsx` spreadsheet. The purpose of this document is to provide transparency about each member's contribution to the project and to support the grading process.

Spreadsheet Format

The spreadsheet must contain a table with the following columns:

- Name
- Faculty Number
- Percentage of contribution
- Tasks Done

Here is an example of a dummy table.

Name	Faculty Number	Percentage	Topics / Tasks Done
Alice	20000001	25%	Implemented BFS and DFS solvers; designed and tested all board levels
Bob	20000002	25%	Implemented the <code>Block</code> class and rolling mechanics; wrote complexity analysis for the report
Charlie	20000003	25%	Implemented the <code>Board</code> and <code>Game</code> classes; added UI and move counter bonus feature
Diana	20000004	25%	Implemented solver with Manhattan distance heuristic; wrote report sections 1–3 and managed submission

Guidelines

- The percentages must sum to exactly **100%** across all group members.
- Contributions need not be equal, but significant imbalances should be clearly explained in the **Topics / Tasks Done** column.
- In cases of suspected dishonesty or significant disagreement among group members, the instructors reserve the right to conduct additional individual assessments.

Important Notes:

- We should be able to run your code through executing the Python file. It is **your responsibility to ensure the code you submit works**, and if we are required to fix it in order to run it, your grade will be penalized. Remember to always test your submission.
- If you have any questions, feel free to send an email to the responsible teachers: Paulo Souza, Pedro Fernandes, and Diogo Rasteiro. Please ensure that **only one member per group is designated as the point of contact for all communications**, as this helps avoid duplicate messages and keeps the exchange clear and organized.